

클라우드 인프라 교육 · 1일차 오후

클라우드 서비스 분석

가상화 · 컨테이너 · 오케스트레이션 · 서버리스 · 클라우드 네이티브

제1장 가상화 — 클라우드의 뿌리

제2장 컨테이너와 Docker

제3장 오케스트레이션과 쿠버네티스

제4장 서버리스

제5장 클라우드 네이티브

제6장 주요 클라우드 서비스

오늘 오후의 목표

- **큰 틀 잡기** — 가상화 → 컨테이너 → 오케스트레이션 → 서버리스로 이어지는 기술의 흐름을 그림으로 이해한다.
- **개념의 이유** — 각 기술이 "어떤 문제를 풀기 위해" 나왔는지를 중심으로 본다. 명령어·세부 설정은 내일 실습에서
- **이론의 완성** — 클라우드 네이티브라는 이름 아래 이 기술들이 어떻게 하나로 묶이는지 정리한다.

오늘의 지도 — 기술은 문제를 풀며 진화했다



각 단계는 이전 단계의 "아쉬움"을 해결하며 등장 — 이 흐름만 잡으면 오늘 오후 전체가 연결된다

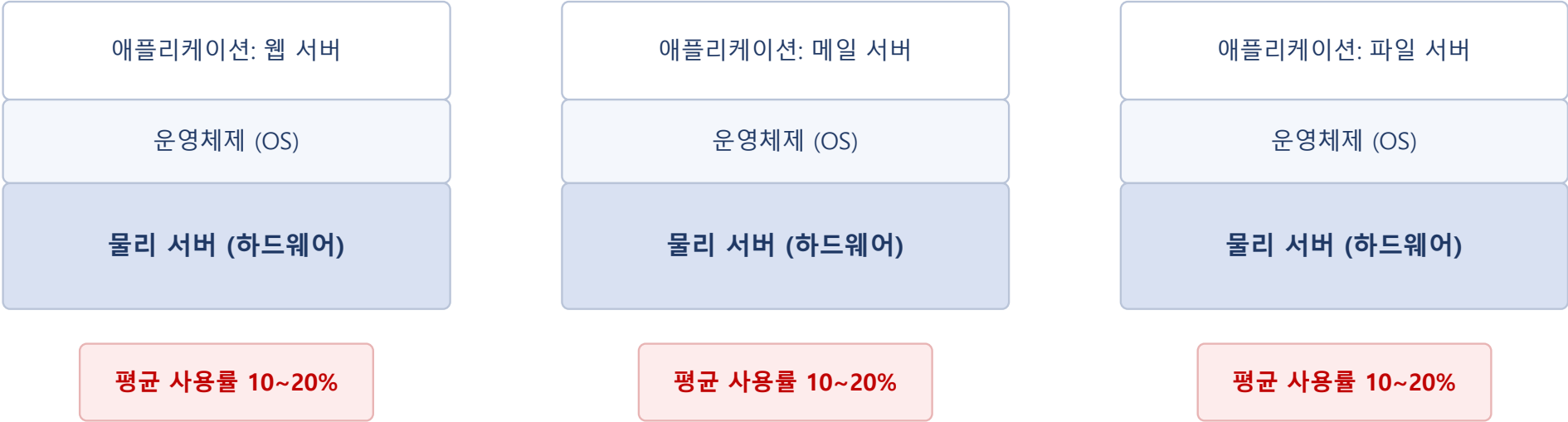
제1장

가상화 — 클라우드의 뿌리

하이퍼바이저 · 가상 머신 · 핵심 기능 · 종류 · 장단점

1.1 가상화 이전 — 물리 서버의 시대

"서버 1대 = 서비스 1개"가 원칙이던 시절



서버는 피크(최대 부하) 기준으로 구매 → 평소에는 80~90%가 놀고 있는 구조. 서버가 늘수록 전기·공간·관리 인력도 함께 증가

물리 서버 방식의 4가지 문제

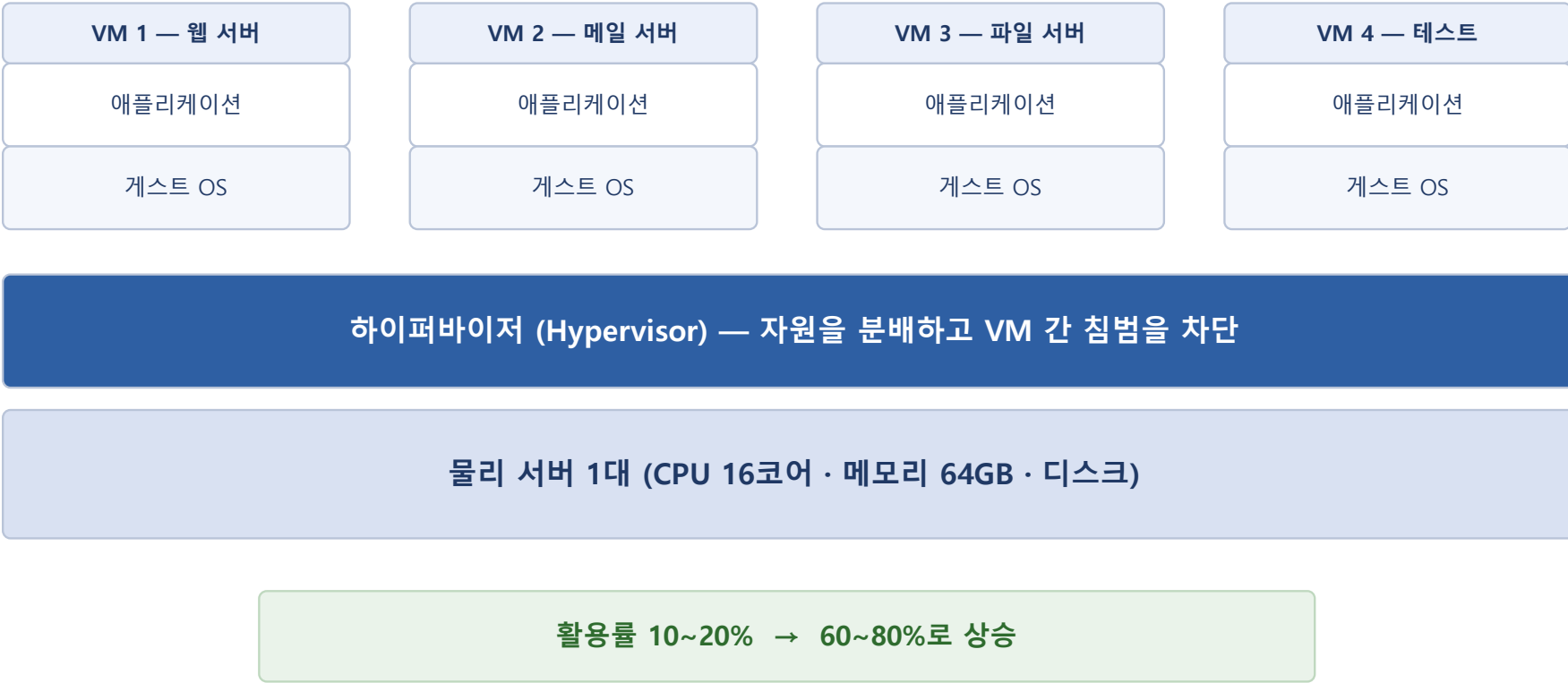
- **낮은 활용률** — 안정성을 위해 서버 1대에 서비스 1개만 운영 → 자원 대부분이 유휴 상태
- **느린 준비 속도** — 새 서비스마다 서버 구매·설치·설정에 수 주 소요
- **관리 부담 증가** — 서버 대수만큼 전기·냉방·상면·교체·패치 업무가 비례 증가
- **복구의 어려움** — 하드웨어 장애 시 동일 장비를 구해 처음부터 재구성해야 함

1.2 가상화(Virtualization)란

한 대의 물리 서버를 소프트웨어 기술로 나누어
여러 대의 독립된 가상 컴퓨터(VM)처럼 사용하는 기술

- 각 VM은 자기만의 OS를 갖고 서로 완전히 격리되어 동작
- 물리 자원(CPU·메모리·디스크)을 필요한 만큼 잘라서 배분

가상화의 구조 — 한 대가 네 대가 되는 원리



서버 4대 살 일을 1대로 해결 — 이 구조가 그대로 확대된 것이 클라우드(EC2)다

가상화의 본질 — 3가지 키워드

- **추상화(Abstraction)** — 소프트웨어가 하드웨어를 흉내 내어, VM은 자신이 진짜 컴퓨터 위에 있다고 "믿고" 동작한다
- **격리(Isolation)** — VM끼리는 서로의 존재를 모른다. 한 VM의 장애·재부팅·해킹이 옆 VM에 번지지 않는다
- **풀링(Pooling)** — 물리 자원을 하나의 풀로 모아 두고 필요한 만큼 떼어 쓴다. 남은 자원의 낭비가 사라진다
- **클라우드와의 관계** — 이 3가지가 어제 배운 "리소스 풀링·멀티테넌시"의 기술적 실체다

가상화 기술의 발전 한눈에

시기	사건	의미
1960~70년대	메인프레임 시분할	자원 공유 개념의 시초
1999년	VMware x86 가상화 상용화	일반 서버 가상화 시대 개막
2000년대	서버 통합 확산	기업 전산실의 표준 기술로
2006년	AWS EC2 출시	가상화의 클라우드 상품화
현재	하드웨어 지원 가상화 표준	CPU 차원의 가상화 지원(Intel VT-x 등)으로 성능 문제 해소

1.3 하이퍼바이저(Hypervisor)란

**물리 자원을 각 VM에 나눠 주고, VM들이 서로 침범하지 못하게 막는
가상화의 핵심 소프트웨어**

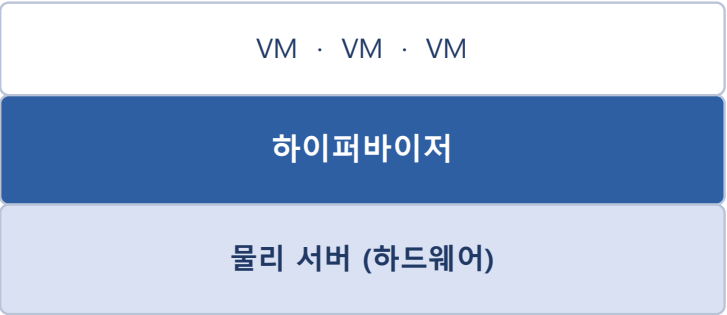
- 가상 머신 모니터(VMM)라고도 부른다
- 하이퍼바이저가 없으면 가상화도 없다 — 가상화 기술의 심장

하이퍼바이저가 하는 일

- **자원 분배** — "VM 1에 4코어·16GB, VM 2에 2코어·8GB"처럼 물리 자원을 배분
- **CPU 스케줄링** — 여러 VM이 CPU를 요청하면 시간을 잘게 쪼개 번갈아 할당
- **격리 유지** — 각 VM의 메모리 영역을 분리해 서로 읽거나 쓸 수 없게 차단
- **명령 중계** — VM의 하드웨어 요청(디스크 쓰기 등)을 받아 실제 하드웨어에 전달
- **상태 관리** — VM의 시작·중지·이동·복제를 제어

하이퍼바이저의 두 유형

Type 1 — 베어메탈형

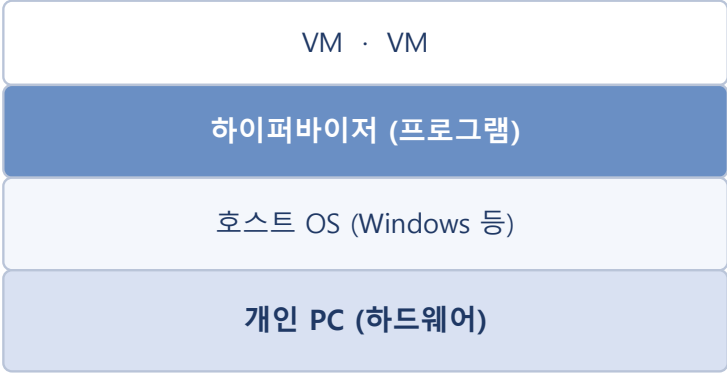


하드웨어 위에 직접 설치 · 고성능·고안정

VMware ESXi · KVM · Hyper-V

데이터센터·클라우드의 표준 (EC2의 기반)

Type 2 — 호스티드형



일반 OS 위에 프로그램처럼 설치

VMware Workstation · VirtualBox

개인 학습·테스트 용도

구분 기준은 "하이퍼바이저가 어디에 설치되는가" — 하드웨어 바로 위(Type 1) vs OS 위(Type 2)

Type 1 vs Type 2 정리

구분	Type 1 (베어메탈)	Type 2 (호스티드)
설치 위치	하드웨어 바로 위	호스트 OS 위
성능	높음 (중간 계층 없음)	상대적으로 낮음
용도	데이터센터·클라우드	개인 PC 학습·테스트
대표 제품	VMware ESXi, KVM, Hyper-V	VMware Workstation, VirtualBox
본 교육 연결	AWS EC2의 기반 기술	집에서 실습 환경 만들 때

1.4 가상 머신(VM)이란

하이퍼바이저가 만들어 낸, 소프트웨어로 구현된 한 대의 완전한 컴퓨터

- 진짜 컴퓨터처럼 CPU·메모리·디스크·네트워크 카드를 (가상으로) 모두 갖추
- 그 위에 OS를 설치하고 프로그램을 실행 — 사용자 입장에선 일반 서버와 동일

VM의 내부 구조



VM 구성 요소 자세히

- **vCPU** — 물리 CPU 코어의 시간을 쪼개 할당받은 가상 CPU. "2 vCPU"는 코어 2개 분량의 연산 시간
- **가상 메모리** — 물리 메모리에서 떼어 배정. 하이퍼바이저가 VM 간 메모리 접근을 차단
- **가상 디스크** — 실체는 스토리지에 저장된 하나의 큰 파일(디스크 이미지). 이 점이 매우 중요하다 (다음 장)
- **가상 NIC** — 가상 네트워크 카드. 가상 스위치를 거쳐 다른 VM·외부와 통신
- **게스트 OS** — VM 안에 설치되는 독립 OS. 물리 서버의 OS(호스트)와 별개

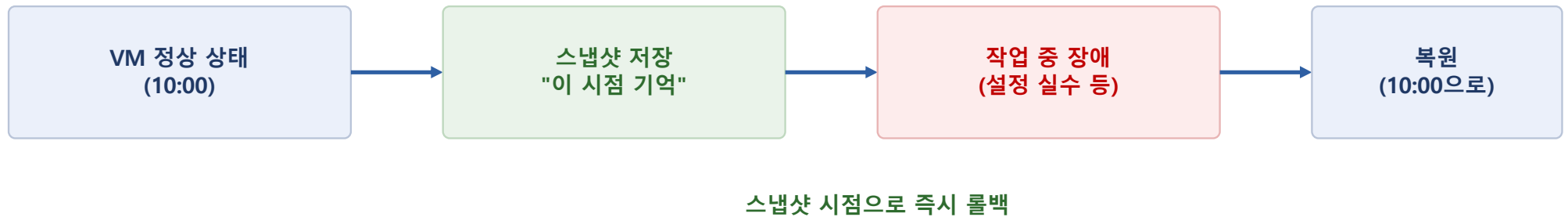
핵심 통찰 — "VM은 결국 파일이다"

**VM의 디스크는 스토리지 위의 파일이다
→ 파일이므로 복사·이동·보관이 가능하다**

- 복사하면 → 동일한 서버가 하나 더 (템플릿·복제)
- 다른 서버로 옮기면 → 서버 이사 (마이그레이션)
- 특정 시점을 저장하면 → 스냅샷 (되돌리기)
- 이 성질에서 가상화의 핵심 기능들이 전부 나온다

1.5 핵심 기능 ① 스냅샷(Snapshot)

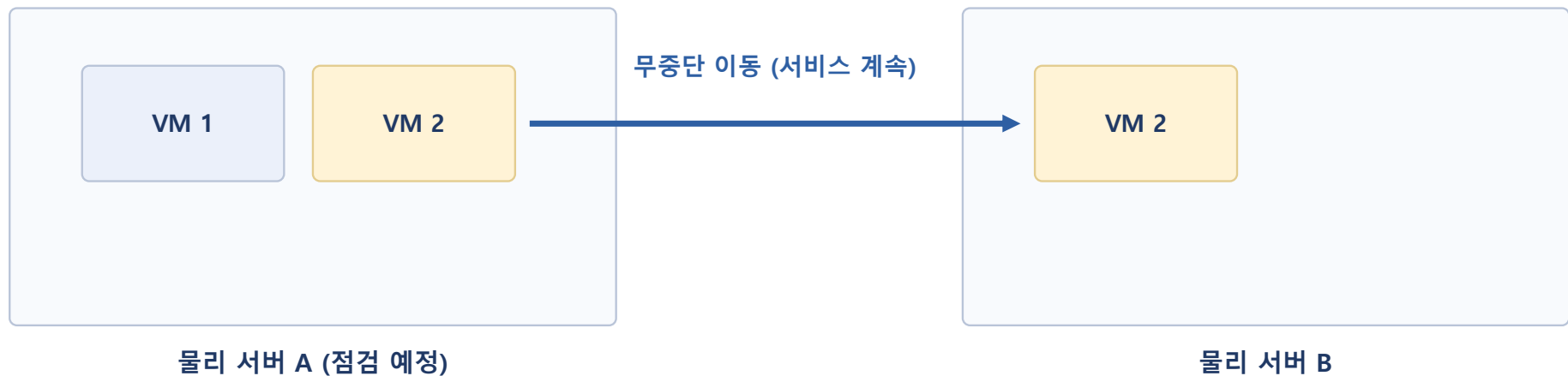
특정 시점의 VM 상태를 저장해 두고, 언제든지 그 시점으로 복원



패치·업그레이드 전 스냅샷은 운영의 기본 수칙 — AWS에서는 EBS 스냅샷·AMI로 같은 개념 사용

핵심 기능 ② 라이브 마이그레이션(Live Migration)

실행 중인 VM을 끄지 않고 다른 물리 서버로 이동

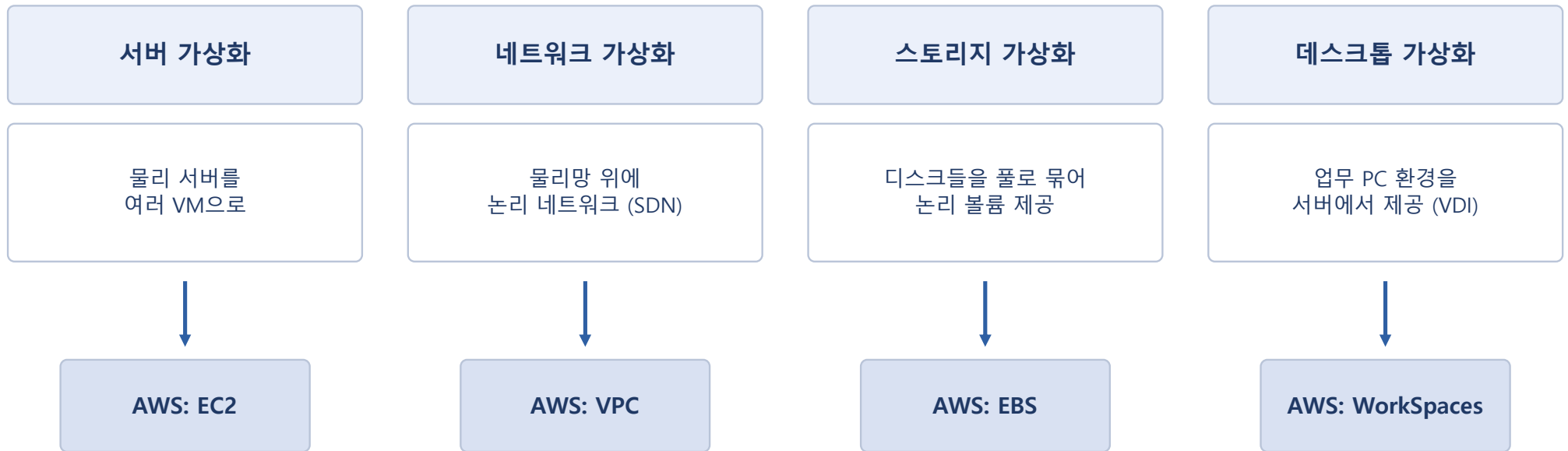


하드웨어 점검·부하 분산 시 서비스를 멈추지 않는 비결 — "VM은 파일"이기에 가능한 일

핵심 기능 ③④ — 템플릿과 고가용성(HA)

- **템플릿(Template)** — 잘 구성한 VM을 틀(원본)로 저장 → 동일한 서버를 몇 대든 복제 생성. AWS의 AMI가 바로 이 개념
- **왜 강력한가** — "서버 설치 문서"가 아니라 "완성된 서버 그 자체"를 복제하므로 구성 실수가 없다
- **고가용성(HA)** — 물리 서버 장애를 감지하면, 그 위의 VM들을 다른 서버에서 자동으로 재기동
- **클라우드 연결** — EC2 인스턴스 장애 시 자동 복구, Auto Scaling의 서버 교체가 모두 이 원리의 확장

1.6 가상화의 종류 — 서버만 가상화되는 것이 아니다



오전에 배운 AWS 핵심 서비스들이 각 가상화 기술의 상품화 — 개념과 서비스가 1:1로 연결된다

VM과 컨테이너는 경쟁이 아니라 보완 관계

실무의 표준 조합 — 내일 실습 구성 그대로



클라우드에서 컨테이너를 쓴다 = 대부분 "VM(EC2) 위에서 Docker를 돌린다"는 뜻 — 두 기술이 층으로 공존

1.7 VM의 장점과 한계

장점

- 강력한 격리 — OS 수준의 완전한 분리
- 이기종 OS 혼용 — 한 서버에 Windows+Linux
- 성숙한 기술 — 수십 년 검증, 도구·인력 풍부
- 기존 시스템 이전 용이 — 물리 서버를 그대로 VM으로
- 스냅샷·복제·이동 — 운영 유연성

한계

- 무겁다 — VM마다 OS 전체 포함, 수 GB
- 느리다 — 부팅에 수 분
- 자원 중복 — VM 10개면 OS도 10개 (메모리·디스크 낭비)
- 배포 단위로 부적합 — 앱 하나 옮기는데 OS까지 통째로
- → 이 한계가 컨테이너 등장 of 직접적 배경

제1장 정리 — 가상화에서 클라우드로

가상화는 클라우드의 뿌리다
AWS = 가상화를 전 세계 규모로 자동화·상품화한 것

- 하이퍼바이저 → EC2 / 템플릿 → AMI / 스냅샷 → EBS 스냅샷
- 네트워크 가상화 → VPC / 스토리지 가상화 → EBS
- 그러나 VM은 무겁다 — 다음 장에서 그 해답(컨테이너)을 본다

제1장 점검

- ☐ 물리 서버 시대의 문제 4가지를 말할 수 있는가?
- ☐ 가상화의 3대 본질(추상화·격리·풀링)을 설명할 수 있는가?
- ☐ Type 1과 Type 2 하이퍼바이저를 그림으로 그릴 수 있는가?
- ☐ "VM은 파일이다"에서 어떤 기능들이 나오는지 연결할 수 있는가?
- ☐ 가상화 4종류와 AWS 서비스의 대응을 말할 수 있는가?
- ☐ VM의 한계 중 컨테이너 등장의 배경이 된 것은?

제2장

컨테이너와 Docker

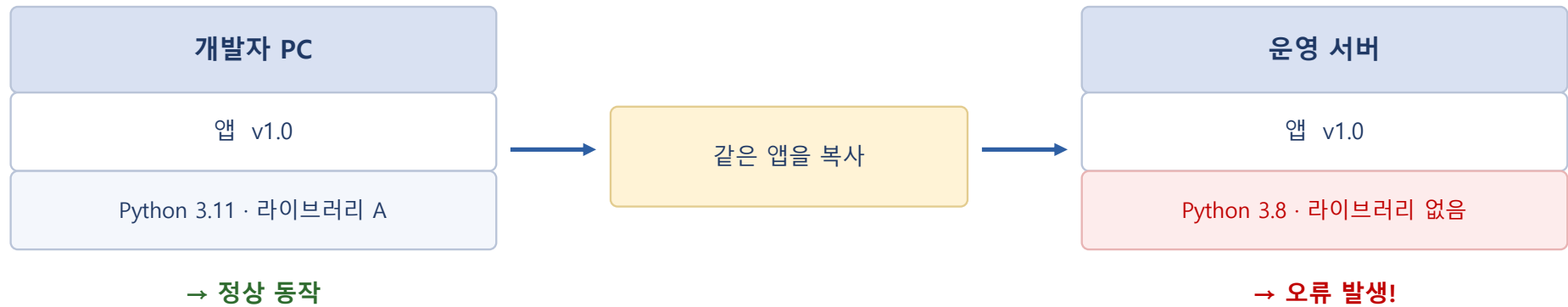
VM의 한계를 넘는 가벼운 격리 — 이론과 큰 그림

2.1 VM의 아쉬움에서 출발

- **상황** — 앱 하나를 다른 서버에서 실행하고 싶을 뿐인데
- **VM의 방식** — OS 전체(수 GB)를 담아 옮기고, 부팅에 수 분을 기다려야 한다
- **질문** — "OS는 그대로 두고, 앱과 그 실행 환경만 딱 떼어 옮길 수는 없을까?"
- **답** — 그것이 컨테이너다

또 하나의 고질병 — 환경 불일치 문제

"제 컴퓨터에서는 되는데요?"



원인: 앱은 같아도 "실행 환경(OS 버전·라이브러리·설정)"이 다르기 때문
해법: 앱과 실행 환경을 통째로 하나의 상자에 포장하자 → 컨테이너

2.2 컨테이너(Container)란

**OS를 새로 설치하지 않고, 호스트 OS의 커널을 함께 쓰면서
앱 + 실행 환경만 격리해 포장한 실행 단위**

- 크기 수십~수백 MB (VM은 수 GB) / 시작 1초 이내 (VM은 수 분)
- 어디서 실행해도 상자 안 환경이 동일 → 환경 불일치 문제 해결

구조 비교 — VM vs 컨테이너

가상 머신 방식



OS가 3번 중복 → 무겁고 느림

컨테이너 방식



OS는 하나를 공유 → 가볍고 빠름

핵심 차이는 "게스트 OS의 유무" — 컨테이너는 OS를 새로 엮지 않고 커널을 빌려 쓴다

VM vs 컨테이너 정리

구분	가상 머신(VM)	컨테이너
격리 단위	하드웨어 수준 (OS 포함)	프로세스 수준 (앱+환경)
크기	수 GB	수십~수백 MB
시작 속도	분 단위	초 단위
집적도	서버당 수~수십 개	서버당 수십~수백 개
격리 강도	강함	VM보다 약함 (커널 공유)
관계	대체가 아니라 보완	실무는 "VM 위에 컨테이너" 조합 (EC2+Docker)

컨테이너의 정체 — 특별한 프로세스

**컨테이너는 미니 컴퓨터가 아니라
"격리된 공간에서 도는, 자원이 제한된 프로세스"다**

- 리눅스 커널 기능으로 프로세스의 시야(파일·네트워크)를 분리하고 자원 사용량을 제한
- OS 부팅이 없으니 1초 만에 시작되는 것

2.3 Docker란

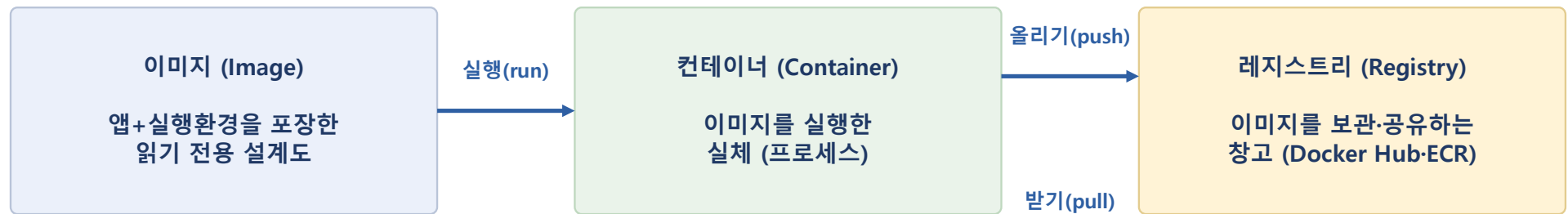
**컨테이너를 만들고(Build) · 옮기고(Ship) · 실행하는(Run)
사실상의 표준 도구**

- 2013년 공개 — 컨테이너 기술 자체는 이전에도 있었으나, Docker가 "이미지"라는 포장·유통 체계로 대중화
- 오늘은 개념의 큰 그림만 — 내일 오후 하루 종일 손으로 실습

컨테이너와 Docker의 등장 배경

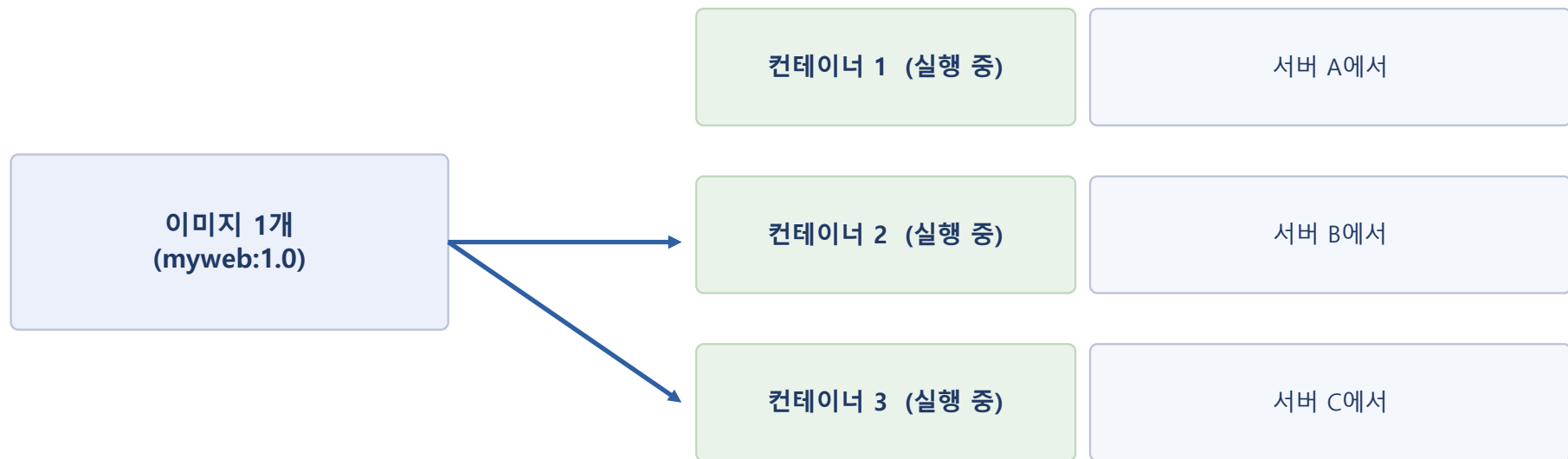
- **기술의 뿌리** — 프로세스 격리 기술은 리눅스에 오래전부터 존재 (chroot, LXC 등)
- **2013년 Docker** — 흩어져 있던 격리 기술을 "이미지로 포장하고 레지스트리로 유통"하는 체계로 묶어 공개
- **왜 폭발했나** — 개발자가 명령 몇 개로 쓸 수 있는 쉬운 도구 + 환경 불일치라는 만성 고통의 해결
- **이후** — 2014년 Kubernetes 공개, 2015년 표준화(OCI) — 컨테이너가 배포의 표준 단위로 정착

Docker의 핵심 3요소 — 이미지 · 컨테이너 · 레지스트리



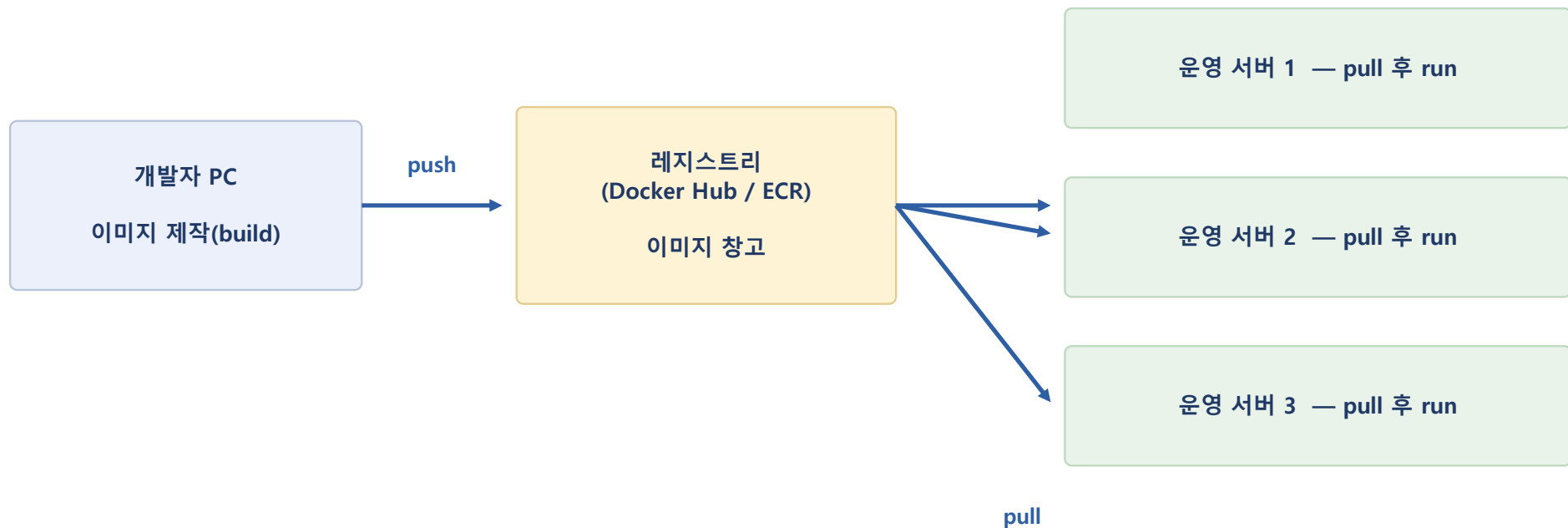
설계도(이미지) 하나로 → 동일한 실체(컨테이너)를 어디서든, 몇 개든 / 창고(레지스트리)로 전 세계 어디로든 배송

이미지와 컨테이너의 관계 — 1 : N



같은 이미지 → 언제나 같은 결과. 트래픽이 늘면 같은 이미지로 컨테이너만 늘리면 된다 (수평 확장과 직결)

컨테이너 배포의 큰 그림 — 만들고, 올리고, 받아서, 실행한다



한 번 만들면(Build once) 어디서나 실행(Run anywhere) — 이 유통 구조가 Docker가 세상을 바꾼 방식

컨테이너 배포의 표준 흐름 (개념만)

내일 오후, 이 4단계를 전부 직접 해 본다

1

만들기 — 앱과 실행 환경을 이미지로 포장 (build)

2

올리기 — 이미지를 레지스트리에 업로드 (push)

3

받기 — 운영 서버가 레지스트리에서 다운로드 (pull)

4

실행 — 어느 서버든 동일하게 컨테이너 실행 (run)

2.4 컨테이너의 장점과 한계

장점

- 환경 일관성 — "내 PC에선 되는데" 문제 소멸
- 가볍고 빠름 — 초 단위 시작, 높은 집적도
- 이식성 — 온프레미스·AWS·Azure 어디든 동일 (벤더 종속 완화)
- 배포 표준화 — 언어·팀이 달라도 포장 방식은 하나
- MSA·CI/CD의 기술적 기반 (제5장 연결)

한계·주의

- 격리 강도 — 커널 공유로 VM보다 약함
- 데이터 — 컨테이너 삭제 시 내부 데이터 소멸 → 외부 저장 필요
- 이미지 보안 — 출처 불명 이미지 위험
- 많아지면 관리 곤란 → 오케스트레이션 필요 (제3장)

컨테이너는 어디에 쓰이나 — 활용 사례

- **애플리케이션 배포** — 웹·API 서버 배포의 사실상 표준 포장 방식
- **개발 환경 통일** — 신규 입사자도 이미지 하나로 즉시 동일 환경 확보
- **MSA 구현** — 잘게 나뉜 서비스들을 각각 컨테이너로 포장 (제5장)
- **클라우드 이식성** — 온프레미스 ↔ 클라우드, 클라우드 ↔ 클라우드 이동의 공통 단위
- **AWS에서는** — ECS·EKS(컨테이너 실행), ECR(레지스트리) — 내일 ECR을 직접 사용

제2장 점검

- ☐ 환경 불일치 문제가 왜 생기고, 컨테이너가 어떻게 해결하는지 그림으로 설명할 수 있는가?
- ☐ VM과 컨테이너 구조 그림에서 결정적 차이(게스트 OS 유무)를 짚을 수 있는가?
- ☐ 이미지·컨테이너·레지스트리의 관계를 한 문장씩으로 말할 수 있는가?
- ☐ 이미지 1개로 컨테이너 여러 개가 가능한 것이 확장에 왜 유리한가?
- ☐ 컨테이너의 한계 중 오케스트레이션이 필요한 이유는?

제3장

오케스트레이션과 쿠버네티스

컨테이너가 많아지면 생기는 일 — 큰 그림만

3.1 컨테이너가 수십 개가 되면



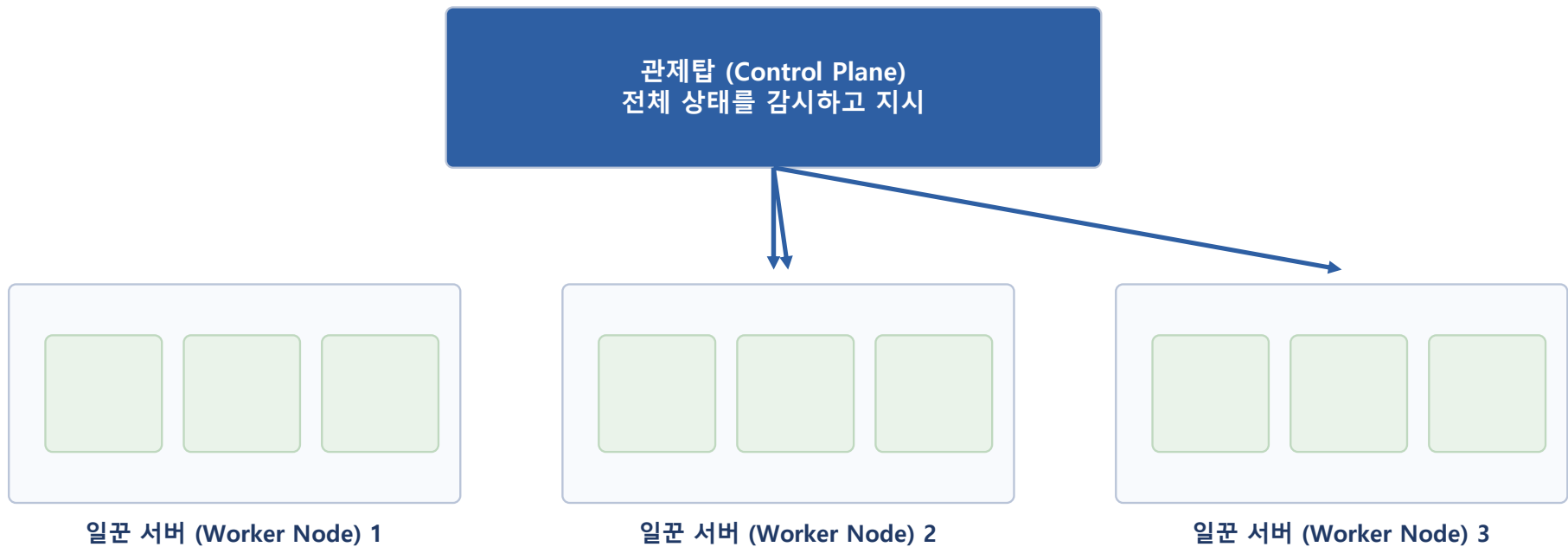
어느 서버에 새 컨테이너를 놓지? · 죽은 컨테이너(X)는 누가 다시 띄우지?
트래픽이 몰리면 누가 늘리지? · 버전 교체는 어떻게 무중단으로 하지?

오케스트레이션(Orchestration)이란

수십~수백 개의 컨테이너를 여러 서버에 걸쳐
자동으로 배치·복구·확장·교체해 주는 관리 기술

- 사람이 컨테이너를 하나하나 챙기는 것은 수십 개부터 불가능
- 이 관리를 자동화하는 표준 도구가 쿠버네티스(Kubernetes, K8s) [Docker Swarm, Nomad 등]

3.2 쿠버네티스의 큰 그림 — 관제탑과 일꾼들



관제탑이 두뇌, 일꾼 서버들이 손발 — 컨테이너는 일꾼 위에서 실행되고, 배치·감시·복구는 관제탑이 담당

쿠버네티스가 자동으로 해 주는 일

- **배치** — 자원 여유를 계산해 새 컨테이너를 알맞은 서버에 배치
- **복구 (자가 치유)** — 죽은 컨테이너를 감지해 자동으로 다시 띄움
- **확장** — 부하가 오르면 컨테이너 수를 늘리고, 내리면 줄임
- **무중단 교체** — 새 버전을 하나씩 올리고 구 버전을 하나씩 내려 서비스 중단 없이 업데이트
- **요컨대** — 사람이 하던 야간 장애 대응·수동 증설을 시스템이 24시간 대신한다

오케스트레이션이 있고 없과의 차이

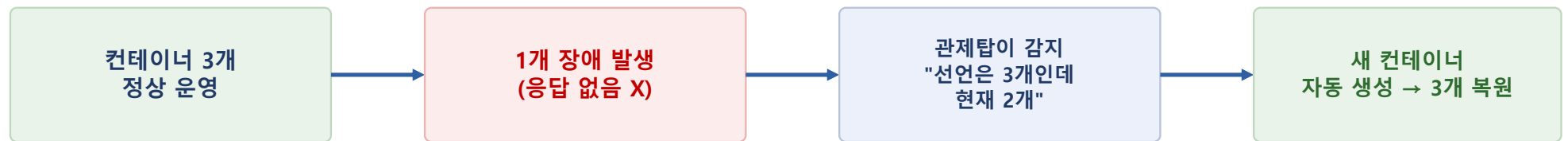
없을 때 (수동 운영)

- 새벽 장애 → 사람이 일어나 재시작
- 트래픽 급증 → 사람이 서버 접속해 증설
- 버전 교체 → 점검 공지 걸고 중단 배포
- 서버 추가 → 어디에 뭘 옮길지 수작업 계산

있을 때 (K8s 자동 운영)

- 장애 감지 → 수 초 내 자동 재생성
- 부하 감지 → 자동 증설, 안정되면 자동 축소
- 무중단 롤링 교체, 문제 시 자동 롤백
- 자원 계산·배치 전부 자동

자가 치유(Self-healing)의 동작



핵심 원리 = 선언적 관리: "3개가 떠 있어야 한다"고 선언해 두면, 시스템이 항상 그 상태를 맞춘다

선언적 관리 — 클라우드 시대의 사고방식

**"이렇게 해라(명령)"가 아니라
"이 상태여야 한다(선언)"로 시스템을 다룬다**

- 선언해 두면 상태 유지가 시스템의 책임이 됨 — 사람은 원하는 상태만 정의

관리형 쿠버네티스와 도입 판단

- **직접 운영의 부담** — 관제탑(Control Plane) 자체의 구축·운영은 상당한 전문성 필요
- **관리형 서비스** — 관제탑을 CSP가 운영해 주는 상품: AWS EKS, Azure AKS, GCP GKE
- **도입 판단 기준** — 컨테이너 소수·단순 구성이면 Docker만으로 충분 / 서비스 다수·잡은 배포·자동 확장이 필요할 때 K8s의 가치 발현

제3장 점검

- ☐ 컨테이너가 많아질 때 생기는 4가지 질문을 말할 수 있는가?
- ☐ 관제탑과 일꾼 서버의 역할 분담을 그림으로 그릴 수 있는가?
- ☐ 자가 치유가 "선언적 관리"에서 어떻게 나오는지 설명할 수 있는가?
- ☐ EKS가 무엇을 대신 운영해 주는 서비스인지 아는가?
- ☐ K8s 도입이 과한 경우와 필요한 경우를 구분할 수 있는가?

여기까지 중간 정리 —

무거운 VM(1장) → 가벼운 컨테이너(2장)
→ 많아진 컨테이너의 자동 관리(3장)

- 각 장은 앞 장의 "아쉬움"에 대한 답이다
- 이 흐름 뒤에 남은 질문: "아예 서버 관리 자체를 없앨 수는 없나?" → 제4장 서버리스

제4장

서버리스 (Serverless)

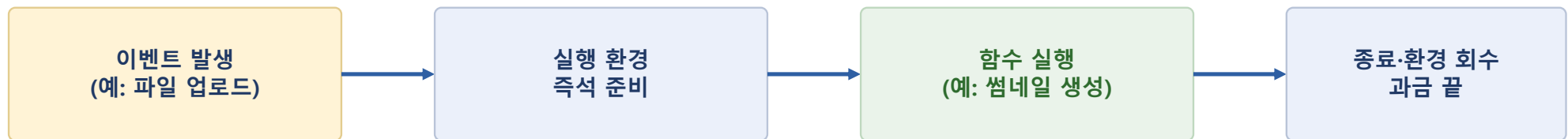
서버 관리가 소멸하는 지점

4.1 서버리스란

**"서버가 없다"가 아니라
"서버를 내가 신경 쓸 필요가 없다"는 뜻**

- 서버는 CSP 어딘가에 있지만, 준비·확장·패치가 전부 자동
- 코드를 함수로 등록 → 요청이 올 때만 실행 → 실행 시간만 과금
- 대표 서비스: AWS Lambda

서버리스의 동작 — 이벤트가 오면 깨어난다



실행된 밀리초만큼만 과금 — 요청이 없으면 비용 0원 / 요청이 폭증하면 자동으로 병렬 확장

서버리스는 어디에 어울리나

- **이벤트 처리** — 파일이 올라오면, 데이터가 도착하면 — "~하면 실행"형 작업
- **정기 작업** — 매일 자정 정산, 주기적 점검처럼 가끔 도는 배치
- **가벼운 API** — 호출 빈도가 낮거나 예측 불가능한 소규모 백엔드
- **어울리지 않는 곳** — 24시간 고부하 상시 서비스(차라리 서버가 저렴), 장시간 연속 작업, 밀리초 지연도 민감한 실시간 처리

서버리스 활용 사례 — 사진 업로드 자동 처리



이 흐름 전체에 관리하는 서버 0대 — 업로드가 없으면 비용도 0원

서버리스의 장점과 제약

장점

- 서버 관리 제로 — 패치·확장·복구 자동
- 유틸 비용 제로 — 안 쓰면 0원
- 자동 병렬 확장
- 작은 기능을 가장 빨리 만드는 방법

제약

- 실행 시간 제한 — 장시간 작업 부적합
- 콜드 스타트 — 오랜만의 첫 호출은 준비 시간으로 지연
- 상시 고부하에는 오히려 비쌈
- CSP 종속성이 상대적으로 큼

선택의 축 — 통제와 편리함의 스펙트럼



정답은 하나가 아니다 — 워크로드마다 스펙트럼 위의 알맞은 위치를 고르는 것이 아키텍처 설계

제4장 점검

- ☐ "서버리스"라는 이름의 진짜 의미를 설명할 수 있는가?
- ☐ 이벤트 → 실행 → 종료 흐름과 과금 방식을 연결할 수 있는가?
- ☐ 서버리스가 어울리는 경우와 아닌 경우를 각각 두 가지씩 말할 수 있는가?
- ☐ VM·컨테이너·서버리스를 스펙트럼 위에 배치할 수 있는가?

제5장

클라우드 네이티브

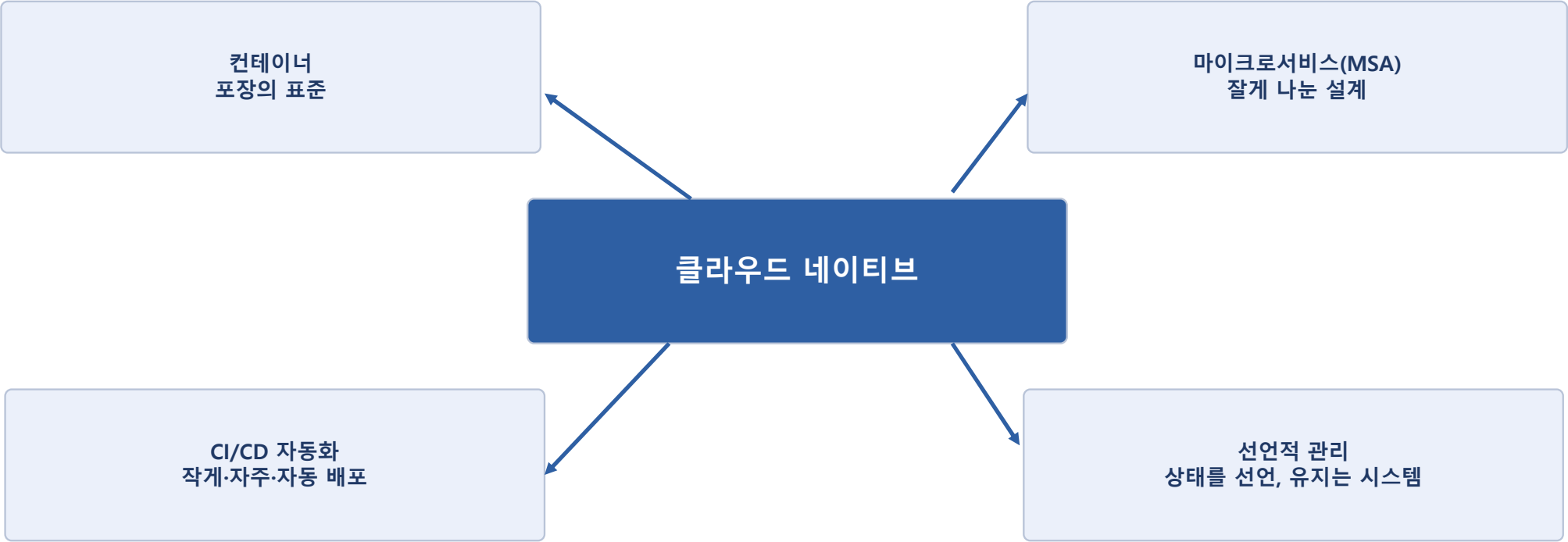
지금까지의 기술이 하나로 모이는 이름

5.1 클라우드 네이티브란

**클라우드에 "올려만 놓은 것"이 아니라
클라우드의 특성(탄력·분산·자동화)을 활용하도록 "만들어진 것"**

- 같은 클라우드 위에서도 만드는 방식에 따라 효과가 하늘과 땅 차이
- 올바른 시스템 구축 방향의 기준이 되는 개념

클라우드 네이티브의 4가지 기둥



오늘 배운 개념들이 전부 이 그림의 부품 — 4개가 함께 갈 때 클라우드의 진짜 효과가 나온다

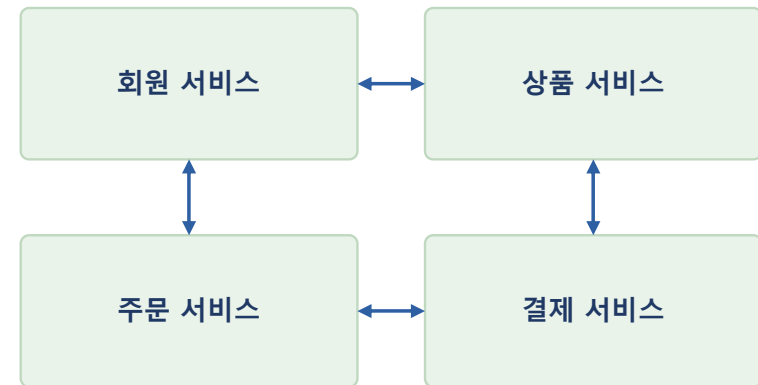
5.2 설계의 전환 — 모놀리식에서 마이크로서비스로

모놀리식 — 한 덩어리



한 줄 수정에도 전체 재배포 · 장애가 전체로 확산

MSA — 독립 서비스들



서비스별 독립 배포·확장 · 장애 국지화 · API로 통신

MSA의 서비스 하나하나를 포장하는 기술이 컨테이너, 그것들을 관리하는 것이 K8s — 세 개념은 한 세트

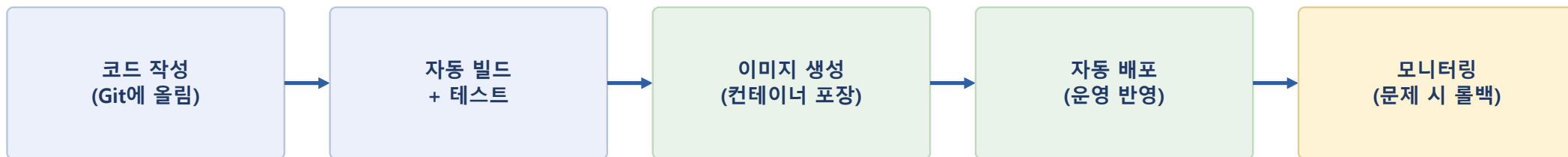
MSA — 개념 수준 정리

- **무엇** — 하나의 큰 애플리케이션을 기능 단위의 작은 독립 서비스로 나누는 설계 방식
- **통신** — 서비스들끼리는 API(오전에 배운 표준 창구)로 대화
- **효과** — 부하가 몰리는 서비스만 골라 확장, 팀별 독립 개발·배포 속도 향상
- **주의 — 만능이 아니다** — 나눈 만큼 관리 복잡도 증가. 소규모 시스템은 모놀리식이 오히려 합리적. "커져서 함께 배포하는 비용이 더 클 때" 가는 방향

5.3 CI/CD — 배포를 자동 파이프라인으로

CI (지속적 통합)

CD (지속적 배포)



사람 손 없이 코드 → 운영까지 자동으로 — "작게, 자주, 자동으로" 배포해 한 번의 변경 위험을 줄인다

DevOps — CI/CD를 가능하게 하는 문화

- **전통 구조의 문제** — 개발팀은 "빨리 배포", 운영팀은 "변경 최소화" → 배포가 분기 1회의 큰 행사가 되고 위험만 누적
- **DevOps** — 개발(Dev)과 운영(Ops)을 하나의 흐름·하나의 책임으로 통합하는 문화·방법론
- **도구가 아니라 방식** — CI/CD 파이프라인은 DevOps 문화를 구현하는 수단
- **클라우드와의 궁합** — 테스트 환경을 API로 즉석 생성·폐기할 수 있는 클라우드에서 진가 발휘

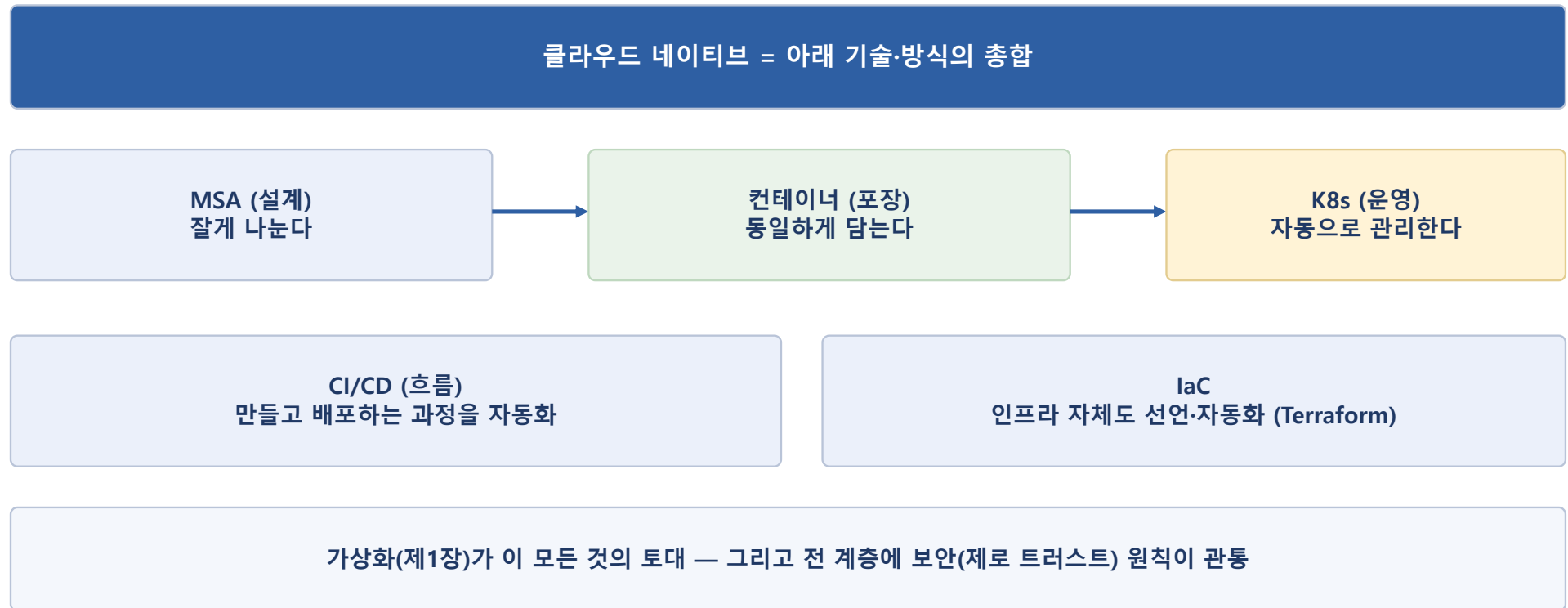
전통 방식 vs 클라우드 네이티브 방식

항목	전통 방식	클라우드 네이티브
애플리케이션 구조	모놀리식 한 덩어리	MSA — 독립 서비스들
배포 단위	서버에 직접 설치	컨테이너 이미지
배포 주기	분기·월 단위 큰 행사	일·주 단위, 작게 자주
배포 방법	수동 절차서	CI/CD 자동 파이프라인
장애 대응	사람이 감지·복구	선언적 관리 — 자동 복구
확장	서버 증설 계획·구매	자동 수평 확장

클라우드 네이티브로 가면 무엇이 좋아지나

- **속도** — 기능 하나를 며칠 만에 운영 반영. 시장 대응 속도가 경쟁력이 됨
- **안정성** — 장애가 국지화되고 자동 복구 — "크게 한 번" 대신 "작게 자주"의 안전함
- **효율** — 필요한 만큼만 자동으로 쓰는 구조 — 어제 배운 탄력성·종량제의 효과 극대화
- **이식성** — 표준 기술(컨테이너·K8s) 기반 — 특정 CSP 종속 완화
- **현실 조언** — 한 번에 전부가 아니라 컨테이너화 → CI/CD → 점진적 MSA 순의 단계적 전환이 정석

전체 연결 — 오늘 배운 것들의 관계 지도



제5장 점검

- ☐ "클라우드에 올린 것"과 "클라우드 네이티브"의 차이를 설명할 수 있는가?
- ☐ 4가지 기둥(컨테이너·MSA·CI/CD·선언적 관리)을 그림으로 그릴 수 있는가?
- ☐ 모놀리식과 MSA의 장단점을 균형 있게 말할 수 있는가?
- ☐ MSA→컨테이너→K8s가 왜 한 세트인지 설명할 수 있는가?
- ☐ CI와 CD가 각각 어느 구간의 자동화인지 구분할 수 있는가?

제6장

주요 클라우드 서비스 정리

오늘 개념이 AWS 서비스로 이어지는 지도

AWS 컴퓨팅 서비스 — 오늘 개념과의 대응

서비스	정체	오늘 배운 개념
EC2	가상 머신(VM) 서비스	하이퍼바이저 위의 VM
ECS / EKS	컨테이너 실행·관리 서비스	컨테이너 + 오케스트레이션
Fargate	서버 관리 없는 컨테이너 실행	컨테이너 × 서버리스의 결합
Lambda	함수 실행 서비스 (FaaS)	서버리스
ECR	컨테이너 이미지 레지스트리	이미지 창고 (내일 실습)

Azure와 GCP — 이름만 다를 뿐 구조는 같다

Microsoft Azure

- VM = Virtual Machines
- 관리형 K8s = AKS
- 서버리스 = Azure Functions
- 강점: MS 제품군 통합, 엔터프라이즈·공공

Google Cloud (GCP)

- VM = Compute Engine
- 관리형 K8s = GKE (K8s 원조 개발사)
- 서버리스 = Cloud Functions
- 강점: 데이터 분석·AI, 컨테이너 기술

서비스 선택의 사고 순서

기술 유행이 아니라 ① 워크로드 특성 → ② 운영 역량 → ③ 통제 필요
순서로 스펙트럼 위의 자리를 정한다

- 간헐 이벤트성 → 서버리스 / 서비스 다수·MSA → 컨테이너 / 레거시·세밀 제어 → VM
- 오전에 배운 Managed-First 원칙과 같은 결

오후 전체 요약 — 한 장의 그림



오늘의 용어 사전 — 한 장 정리

용어	뜻
하이퍼바이저	물리 자원을 VM들에 분배·격리하는 가상화 핵심 소프트웨어
VM (가상 머신)	소프트웨어로 구현된 완전한 컴퓨터 — 실체는 파일
스냅샷 / 템플릿	시점 저장·복원 / 완성 서버의 복제 원본 (AWS: AMI)
컨테이너	커널을 공유하며 앱+환경만 격리한 실행 단위
이미지 / 레지스트리	컨테이너의 설계도 / 이미지를 보관·공유하는 창고 (ECR)
오케스트레이션	다수 컨테이너의 배치·복구·확장 자동화 (K8s)
선언적 관리	원하는 상태를 선언하면 시스템이 유지 — K8s·IaC의 철학
서버리스 / FaaS	이벤트 시에만 함수 실행, 실행 시간만 과금 (Lambda)
MSA	기능 단위의 독립 서비스로 나누는 설계, 통신은 API
CI/CD	빌드·테스트·배포의 자동 파이프라인
클라우드 네이티브	위 요소들을 묶은 클라우드 시대의 구축 방식 총칭

오후 세션 요약

- 가상화: 하이퍼바이저가 물리 서버를 VM으로 분할 — 추상화·격리·폴링. "VM은 파일"이라서 스냅샷·템플릿·이동이 가능하고, 이것이 클라우드(EC2·AMI)의 뿌리
- 컨테이너: OS를 새로 엮지 않고 커널을 공유 — 가볍고 빠르며, 이미지→레지스트리→실행 유통 구조로 "한 번 만들어 어디서나 실행"
- 오케스트레이션: 컨테이너가 많아지면 배치·복구·확장을 자동화 — 관제탑과 일꾼, 선언적 관리와 자가 치유 (K8s/EKS)
- 서버리스: 이벤트가 올 때만 실행하고 실행 시간만 과금 — 서버 관리의 소멸 (Lambda)
- 클라우드 네이티브: MSA(설계)+컨테이너(포장)+CI/CD(흐름)+선언적 관리(운영)의 총합 — 올바른 구축 방향의 기준
- 선택은 스펙트럼: 통제가 필요할수록 VM 쪽, 운영을 덜 하고 싶을수록 서버리스 쪽